

1 Import Libraries

```
import json
import pickle
import numpy as np
import gradio as gr
import tensorflow as tf
from tensorflow.keras.preprocessing.sequence import pad_sequences
import random
```

Explanation of imports:

- **json** → Load the dataset containing intents, patterns, and responses.
- **pickle** → Load the saved tokenizer and class list.
- **numpy (np)** → Handle arrays and numerical operations.
- **gradio (gr)** → Build an interactive web interface for your chatbot.
- **tensorflow (tf)** → Load and run your trained Keras model.
- **pad_sequences** → Make sequences same length for LSTM input.
- **random** → Choose random responses from a list.

2 Load Model and Resources

```
model = tf.keras.models.load_model("shop_bot.keras", compile=False)
tokenizer = pickle.load(open("tokenizer.pkl", "rb"))
classes = pickle.load(open("classes.pkl", "rb"))
```

```
with open("clothes_data.json", "r", encoding="utf-8") as f:
    data = json.load(f)
```

```
max_len = model.input_shape[1]
```

- Loads your **trained LSTM model** (shop_bot.keras).
- Loads **tokenizer** and **class labels** (classes.pkl) used to preprocess text.

- Loads **intent data** with tags and responses.
- `max_len` → The maximum input length expected by the model.

3 Preprocess Input

```
def clean_up_sentence(sentence):  
    return sentence.lower()
```

- Converts the user input to lowercase.
- Simple preprocessing before tokenizing.

4 Predict User Intent

```
def predict_class(sentence):  
    cleaned = clean_up_sentence(sentence)  
    seq = tokenizer.texts_to_sequences([cleaned])  
    padded = pad_sequences(seq, maxlen=max_len, padding="post")  
    pred = model.predict(padded, verbose=0)[0]  
    idx = np.argmax(pred)  
    return classes[idx]
```

Step-by-step:

1. Clean the sentence (lowercase).
2. Convert text → sequence of word indices using **tokenizer**.
3. Pad the sequence to **max_len** for LSTM.
4. Predict probabilities for each class using the model.
5. Get the index of the **highest probability** → predicted class tag.

5 Get Bot Response

```
def get_response(tag):  
    for intent in data["intents"]:  
        if intent["tag"] == tag:  
            return random.choice(intent["responses"])  
    return "Sorry, I didn't understand that."
```

- Looks up the predicted tag in the **intents JSON**.
- Picks a **random response** from the list for variety.
- Default response if tag is not found.

6 Chat Function

```
def chatbot(message, history):  
    tag = predict_class(message)  
    response = get_response(tag)  
    return response
```

- Combines the prediction and response logic.
- Input: user message
- Output: bot response

7 Create Gradio Chat Interface

```
chat_ui = gr.ChatInterface(  
    fn=chatbot,  
    title="👗 Clothing Assistant Bot",  
    description="Ask about outfits, colors, brands, events, sizing &  
more!"  
)
```

- Uses Gradio's **ChatInterface** to make a ready-to-use chat UI.
- `fn=chatbot` → Function that handles chat messages.
- `title` and `description` → Displayed on the interface.

8 Launch the UI

```
if __name__ == "__main__":
    chat_ui.launch(share=True)
```

- Runs the app locally.
- `share=True` → Generates a **public URL** you can share with anyone.

✓ Summary (Flow of Data)

1. User types a message.
2. `chatbot()` → predicts the intent using LSTM model.
3. `get_response()` → selects a random response for that intent.
4. Response is displayed in **Gradio chat interface**.

1 json

```
import json
```

- **Purpose:** Work with JSON files.
- **Used functions:**
 - `json.load(file)` → Reads JSON data from a file into a Python dictionary.
 - **Use in your code:** Load the chatbot dataset (`clothes_data.json`) containing intents, patterns, and responses.

2 pickle

```
import pickle
```

- **Purpose:** Save and load Python objects.
- **Used functions:**
 - `pickle.load(file)` → Load saved Python objects from a file.
 - **Use in your code:** Load the saved **tokenizer** and **classes** (`tokenizer.pkl` and `classes.pkl`) for preprocessing and prediction.

3 numpy (np)

```
import numpy as np
```

- **Purpose:** Handle numerical operations and arrays efficiently.
- **Used functions:**
 - `np.argmax(array)` → Returns the index of the maximum value in an array.
 - **Use in your code:** Determine the class with the highest predicted probability from the LSTM model.

4 gradio (gr)

```
import gradio as gr
```

- **Purpose:** Build interactive web interfaces for machine learning models.
- **Used class/function:**
 - `gr.ChatInterface()` → Creates a ready-to-use chat interface.
 - **Use in your code:** Displays a UI where users can chat with the bot.
- `chat_ui.launch(share=True)` → Starts the Gradio app and optionally shares a public URL.

5 tensorflow (tf)

```
import tensorflow as tf
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

- **Purpose:** Build, load, and run deep learning models.
- **Used functions/classes:**
 - `tf.keras.models.load_model("shop_bot.keras")` → Load a trained Keras model.
 - `pad_sequences()` → Make sequences the same length for LSTM input.
- **Use in your code:** Preprocess user input and predict intent with your LSTM model.

6 random

```
import random
```

- **Purpose:** Perform random operations.
- **Used functions:**
 - `random.choice(list)` → Pick a random element from a list.
 - **Use in your code:** Select a random response from the responses list for the predicted intent, making the bot's replies varied.

Summary Table

Library	Functions/Classes Used	Purpose in Your Code
json	<code>json.load()</code>	Load intents & responses dataset
pickle	<code>pickle.load()</code>	Load saved tokenizer & class list

numpy	<code>np.argmax()</code>	Get index of highest predicted class
gradio	<code>gr.ChatInterface(), launch()</code>	Build and run interactive chat UI
tensorflow	<code>tf.keras.models.load_model(), pad_sequences()</code>	Load LSTM model, preprocess input
random	<code>random.choice()</code>	Pick random response from intent